Homework 2

Tejas Patel

Question 1

Your goal is to predict abalone age, which is calculated as the number of rings plus 1.5. Notice there currently is no `age` variable in the data set. Add `age` to the data set.

Assess and describe the distribution of `age`.

```
# R codewriting in this homework was assisted by Anthropic Claude Opus 5.5
# Everything else is original work
set.seed(174)
library(tidyverse)
library(tidymodels)
library(kknn)
tidymodels_prefer()
abalone <- read_csv("data/abalone.csv") %>%
  mutate(age = rings + 1.5,
         type = factor(type))

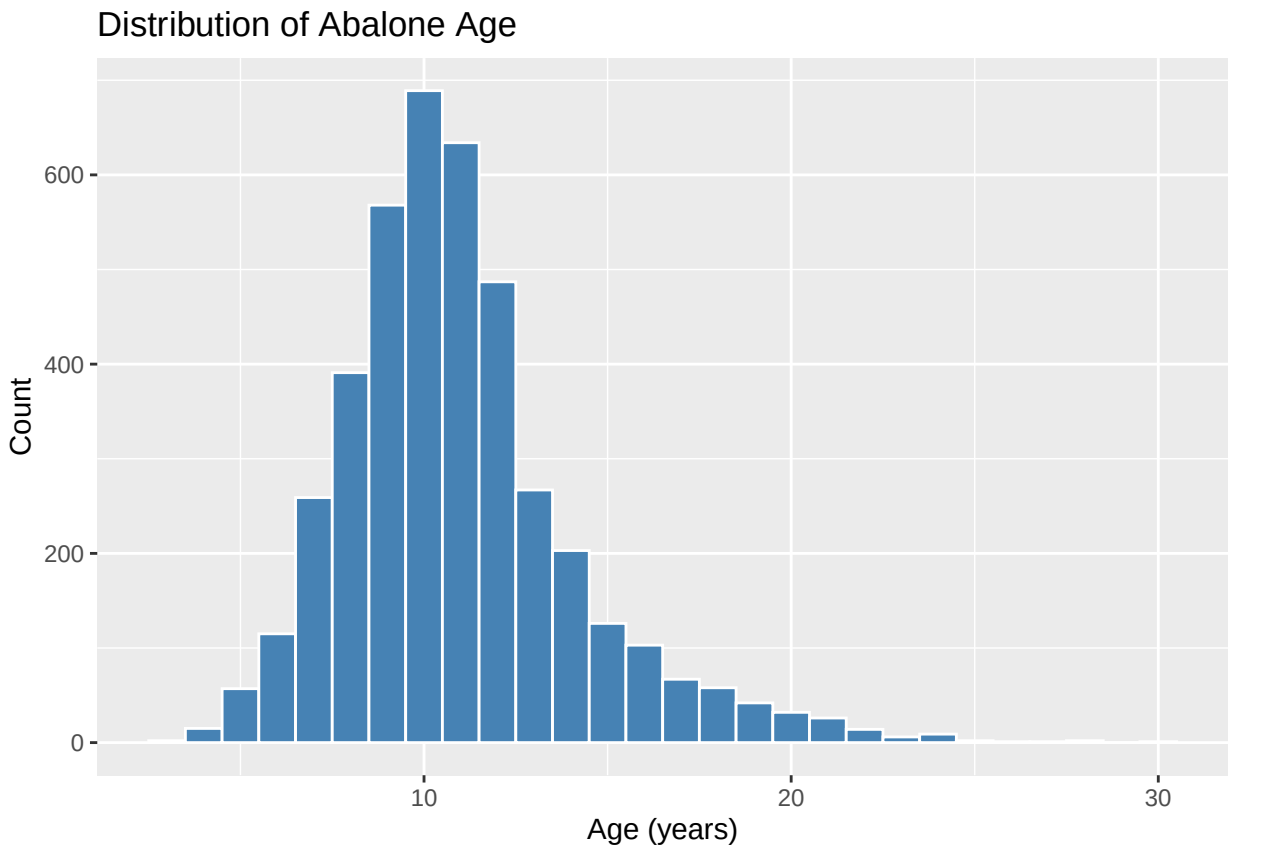
head(abalone)
```

```
## # A tibble: 6 x 10
##   type  longest_shell diameter height whole_weight shucked_weight viscera_weight
##   <fct>         <dbl>    <dbl>  <dbl>        <dbl>          <dbl>          <dbl>
## 1 M             0.455    0.365  0.095        0.514         0.224          0.101
## 2 M             0.35     0.265  0.09         0.226         0.0995         0.0485
## 3 F             0.53     0.42   0.135        0.677         0.256          0.142
## 4 M             0.44     0.365  0.125        0.516         0.216          0.114
## 5 I             0.33     0.255  0.08         0.205         0.0895         0.0395
## 6 I             0.425    0.3    0.095        0.352         0.141          0.0775
## # i 3 more variables: shell_weight <dbl>, rings <dbl>, age <dbl>
```

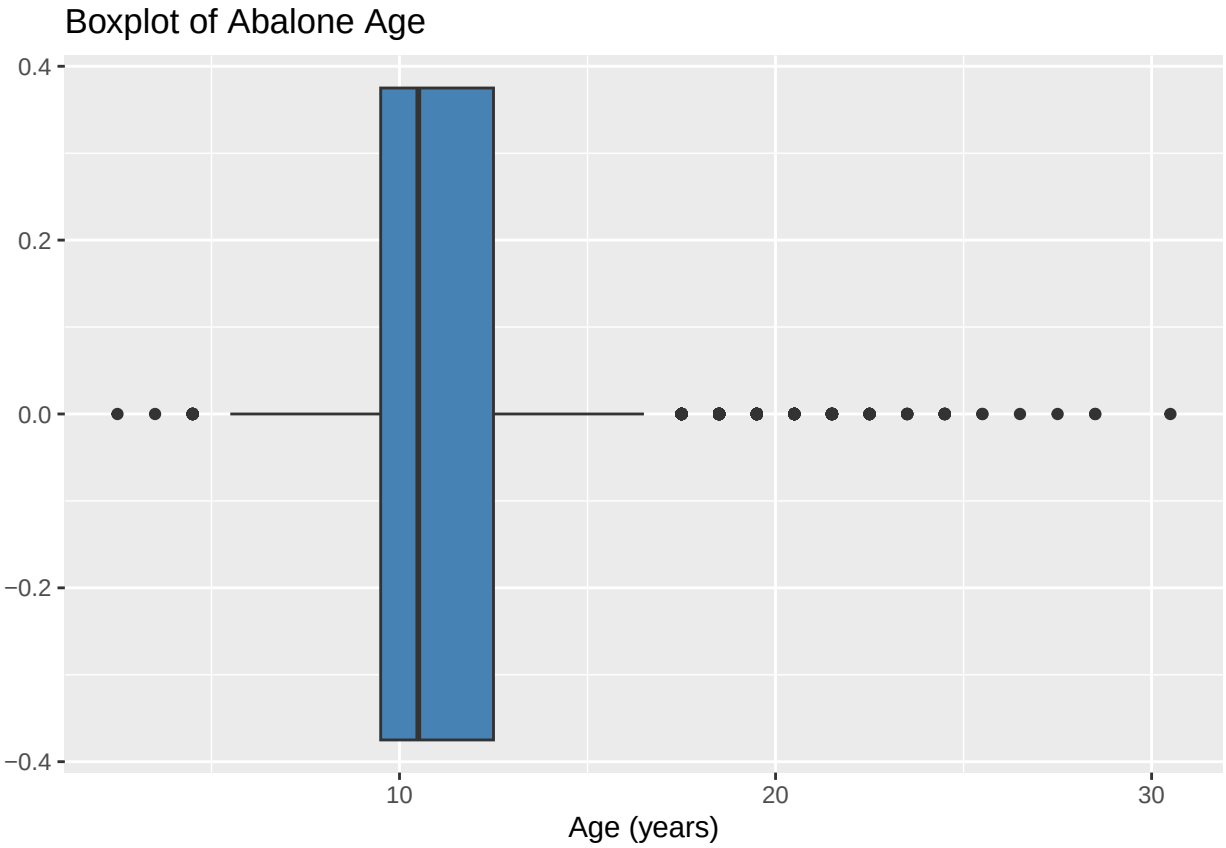
```
abalone %>%
  summarise(n = n(),
            mean = mean(age),
            median = median(age),
            sd = sd(age),
            min = min(age),
            q1 = quantile(age, 0.25),
            q3 = quantile(age, 0.75),
            max = max(age))
```

```
## # A tibble: 1 x 8
##       n  mean median    sd   min    q1    q3   max
##   <int> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1  4177  11.4   10.5  3.22   2.5   9.5  12.5  30.5
```

```r
abalone %>%
  ggplot(aes(x = age)) +
  geom_histogram(binwidth = 1, fill = "steelblue", color = "white") +
  labs(title = "Distribution of Abalone Age",
       x = "Age (years)",
       y = "Count")
```




```r
abalone %>%
  ggplot(aes(x = age)) +
  geom_boxplot(fill = "steelblue") +
  labs(title = "Boxplot of Abalone Age",
       x = "Age (years)")
```



It looks like the `abalone` dataset is right/old-skewed with a mean age of around 11.4 years and a median almost 1 year younger than that. The max is almost 3 times the median, but the 25th and 75th percentiles are only 3 years apart meaning the data is tightly bound around the median. About 6.7% of points are outliers, with the visible blue bar quantities on the chart ending at 24 years old.

Question 2

Split the abalone data into a training set and a testing set. Use stratified sampling. You should decide on appropriate percentages for splitting the data.

```
set.seed(174)

abalone_split <- initial_split(abalone, prop = 0.85, strata = age)
abalone_train <- training(abalone_split)
abalone_test <- testing(abalone_split)

dim(abalone_train)
```

```
## [1] 3548   10
```

```
dim(abalone_test)
```

```
## [1] 629  10
```

I chose an 85/15 split since when I took a course on machine learning from Andrew Ng, *I think* he said 85/15 was the split he uses. 629 test rows is perfectly acceptable and the model should train well on 3548 rows.

Question 3

Using the **training** data, create a recipe predicting the outcome variable, `age`, with all other predictor variables. Note that you **should not** include `rings` to predict `age`. *Explain why you shouldn't use* ***`rings`*** *to predict* ***`age`****.*

Steps for your recipe:

1. dummy code any categorical predictors
2. create interactions between
    - `type` and `shucked_weight`,
    - `longest_shell` and `diameter`,
    - `shucked_weight` and `shell_weight`
3. center all predictors, and
4. scale all predictors.

You'll need to investigate the `tidymodels` documentation to find the appropriate step functions to use.

```
abalone_recipe <- recipe(age ~ type + longest_shell + diameter + height +
                           whole_weight + shucked_weight + viscera_weight +
                           shell_weight,
                         data = abalone_train) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_interact(terms = ~ starts_with("type_"):shucked_weight +
                  longest_shell:diameter +
                  shucked_weight:shell_weight) %>%
  step_center(all_predictors()) %>%
  step_scale(all_predictors())

abalone_recipe
```

```
##
## -- Recipe ----------------------------------------------------------------------
##
## -- Inputs
## Number of variables by role
## outcome:   1
## predictor: 8
##
## -- Operations
## * Dummy variables from: all_nominal_predictors()
## * Interactions with: ...
## * Centering for: all_predictors()
## * Scaling for: all_predictors()
```

```
abalone_recipe %>%
  prep() %>%
  bake(new_data = NULL) %>%
  head()
```

```
## # A tibble: 6 x 14
##   longest_shell diameter height whole_weight shucked_weight viscera_weight
##           <dbl>    <dbl>  <dbl>        <dbl>          <dbl>          <dbl>
## 1        -1.62    -1.54  -1.41        -1.27          -1.22          -1.29
## 2        -0.828   -1.09  -1.05        -0.975         -0.985         -0.943
## 3        -1.41    -1.29  -1.29        -1.10          -1.19          -1.29
## 4        -0.496   -0.537 -0.817       -0.715         -0.599         -0.519
## 5        -0.621   -0.537 -0.817       -0.627         -0.554         -0.587
## 6        -2.66    -2.60  -1.99        -1.61          -1.50          -1.51
## # i 8 more variables: shell_weight <dbl>, age <dbl>, type_I <dbl>,
## #   type_M <dbl>, type_I_x_shucked_weight <dbl>, type_M_x_shucked_weight <dbl>,
## #   longest_shell_x_diameter <dbl>, shucked_weight_x_shell_weight <dbl>
```

You shouldn't use rings to predict age because rings are directly caused by age, and the model can be determined by inverting the age->rings function. It makes the rest of the predictors essentially useless since a known causal relationship is being used to predict age.

Question 4

Create and store a linear regression object using the `"lm"` engine.

```
lm_model <- linear_reg() %>%
  set_engine("lm") %>%
  set_mode("regression")

lm_model
```

```
## Linear Regression Model Specification (regression)
##
## Computational engine: lm
```

Question 5

Create and store a KNN object using the `"kknn"` engine. Specify `k = 7`.

```
knn_model <- nearest_neighbor(neighbors = 7) %>%
  set_engine("kknn") %>%
  set_mode("regression")

knn_model
```

```
## K-Nearest Neighbor Model Specification (regression)
##
## Main Arguments:
##   neighbors = 7
##
## Computational engine: kknn
```

Question 6

Now, for each of these models (linear regression and KNN):

1. set up an empty workflow,
2. add the model, and
3. add the recipe that you created in Question 3.

Note that you should be setting up two separate workflows.

Fit both models to the training set.

```
lm_wflow <- workflow() %>%
  add_model(lm_model) %>%
  add_recipe(abalone_recipe)

knn_wflow <- workflow() %>%
  add_model(knn_model) %>%
  add_recipe(abalone_recipe)
```

```
lm_fit <- fit(lm_wflow, data = abalone_train)
knn_fit <- fit(knn_wflow, data = abalone_train)

lm_fit %>%
  extract_fit_parsnip() %>%
  tidy()
```

```
## # A tibble: 14 x 5
##    term                          estimate std.error statistic  p.value
##    <chr>                            <dbl>     <dbl>     <dbl>    <dbl>
##  1 (Intercept)                     11.5      0.0367   312.    0
##  2 longest_shell                    0.466    0.281      1.66  9.67e- 2
##  3 diameter                         2.20     0.308      7.16  1.00e-12
##  4 height                           0.262    0.0689     3.81  1.43e- 4
##  5 whole_weight                     4.98     0.390     12.8   1.71e-36
##  6 shucked_weight                  -4.26     0.247    -17.3   3.07e-64
##  7 viscera_weight                  -1.02     0.155     -6.54  7.22e-11
##  8 shell_weight                     1.61     0.209      7.72  1.52e-14
##  9 type_I                          -0.919    0.111     -8.26  1.99e-16
## 10 type_M                          -0.223    0.101     -2.21  2.71e- 2
## 11 type_I_x_shucked_weight          0.468    0.0843     5.54  3.16e- 8
## 12 type_M_x_shucked_weight          0.237    0.106      2.23  2.59e- 2
## 13 longest_shell_x_diameter        -2.79     0.394     -7.06  1.94e-12
## 14 shucked_weight_x_shell_weight   -0.149    0.201     -0.742 4.58e- 1
```

```
knn_fit
```

```
## == Workflow [trained] ==========================================================
## Preprocessor: Recipe
## Model: nearest_neighbor()
##
## -- Preprocessor ----------------------------------------------------------------
## 4 Recipe Steps
##
## * step_dummy()
## * step_interact()
## * step_center()
## * step_scale()
##
## -- Model -----------------------------------------------------------------------
##
## Call:
## kknn::train.kknn(formula = ..y ~ ., data = data, ks = min_rows(7,     data, 5))
##
## Type of response variable: continuous
## minimal mean absolute error: 1.654218
```

```
## Minimal mean squared error: 5.598762
## Best kernel: optimal
## Best k: 7
```

Question 7

Use your linear regression `fit()` object to predict the age of a hypothetical female abalone with longest_shell = 0.50, diameter = 0.10, height = 0.30, whole_weight = 4, shucked_weight = 1, viscera_weight = 2, and shell_weight = 1.

```
new_abalone <- tibble(type = factor("F", levels = levels(abalone$type)),
                      longest_shell = 0.50,
                      diameter = 0.10,
                      height = 0.30,
                      whole_weight = 4,
                      shucked_weight = 1,
                      viscera_weight = 2,
                      shell_weight = 1)

predict(lm_fit, new_data = new_abalone)
```

```
## # A tibble: 1 x 1
##   .pred
##   <dbl>
## 1  21.8
```

Question 8

Now you want to assess your models' performance. To do this, use the `yardstick` package:

1. Create a metric set that includes R^2, RMSE (root mean squared error), and MAE (mean absolute error).
2. Use `augment()` to create a tibble of your model's predicted values from the **testing data** along with the actual observed ages (these are needed to assess your model's performance).
3. Finally, apply your metric set to the tibble, report the results, and interpret the *R^2* value.

Repeat these steps once for the linear regression model and for the KNN model.

```
abalone_metrics <- metric_set(rsq, rmse, mae)
```

```
lm_test_results <- augment(lm_fit, new_data = abalone_test) %>%
  select(age, .pred)

head(lm_test_results)
```

```
## # A tibble: 6 x 2
##     age .pred
##   <dbl> <dbl>
## 1   8.5  9.51
## 2  11.5 12.5
## 3  13.5 12.9
## 4   8.5 10.5
## 5  12.5  9.91
## 6  13.5 12.0
```

```
abalone_metrics(lm_test_results, truth = age, estimate = .pred)
```

```
## # A tibble: 3 x 3
```

```
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rsq     standard       0.585
## 2 rmse    standard       1.95
## 3 mae     standard       1.44
```

```
knn_test_results <- augment(knn_fit, new_data = abalone_test) %>%
  select(age, .pred)

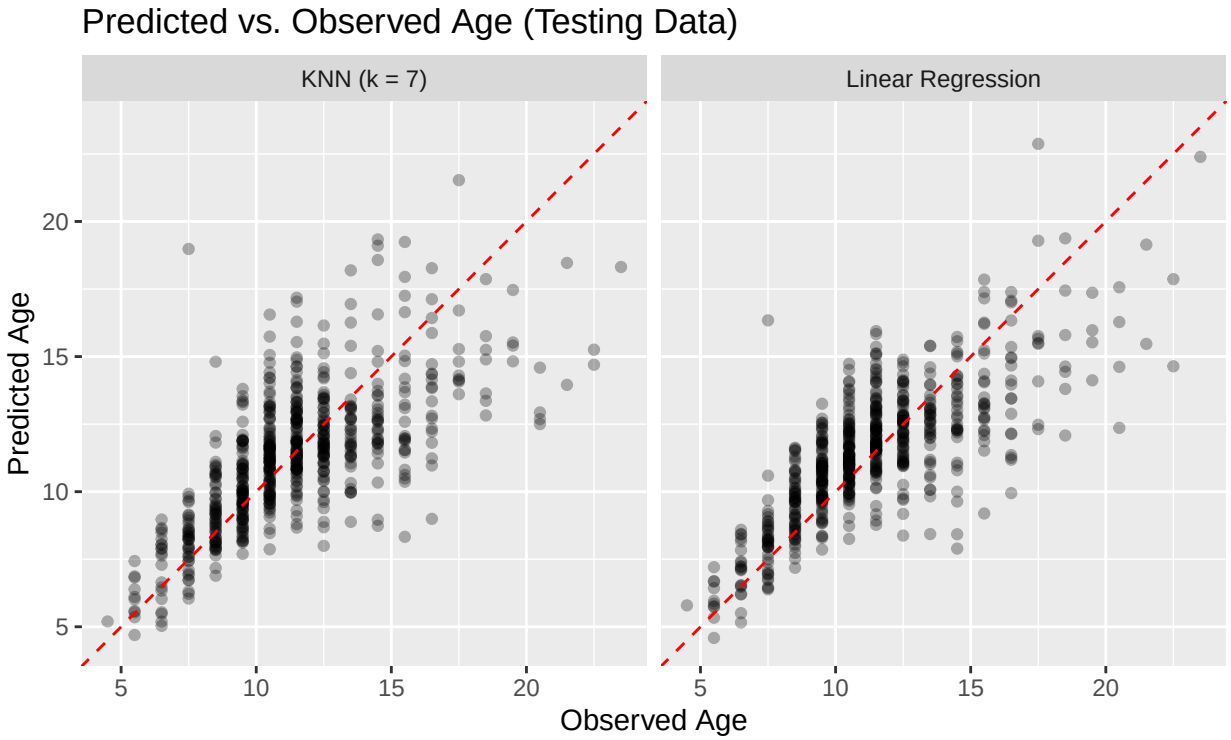
head(knn_test_results)
```

```
## # A tibble: 6 x 2
##     age .pred
##   <dbl> <dbl>
## 1   8.5  9.31
## 2  11.5 12.0
## 3  13.5 13.4
## 4   8.5 10.9
## 5  12.5  8.68
## 6  13.5 11.7
```

```
abalone_metrics(knn_test_results, truth = age, estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rsq     standard       0.483
## 2 rmse    standard       2.21
## 3 mae     standard       1.62
```

```
bind_rows(lm_test_results %>% mutate(model = "Linear Regression"),
          knn_test_results %>% mutate(model = "KNN (k = 7)")) %>%
  ggplot(aes(x = age, y = .pred)) +
  geom_point(alpha = 0.3) +
  geom_abline(linetype = "dashed", color = "red") +
  coord_obs_pred() +
  facet_wrap(~ model) +
  labs(title = "Predicted vs. Observed Age (Testing Data)",
       x = "Observed Age",
       y = "Predicted Age")
```



The Linear Regression model has an R^2 of 0.585, which is higher than the KNN(7) model result of 0.483. Neither is particularly great, but they do capture the essence of the data. The R^2 measures what percent of variance in age is explained by the model, with $1 - R^2$ measuring how much of the variance in age went unexplained. 41.5% of the variance in age went unexplained in the linear model, where 51.7% went unexplained on KNN.

The results: RMSE is lower on the linear model and same with MAE (1.95 vs 2.21 and 1.44 vs 1.62 respectively).

Question 9

Which model performed better on the testing data? Explain why you think this might be. Are you surprised by any of your results? Why or why not?

```
bind_rows(
  abalone_metrics(lm_test_results, truth = age, estimate = .pred) %>%
    mutate(model = "Linear Regression", data = "Testing"),
  abalone_metrics(knn_test_results, truth = age, estimate = .pred) %>%
    mutate(model = "KNN (k = 7)", data = "Testing"),
  augment(lm_fit, new_data = abalone_train) %>%
    abalone_metrics(truth = age, estimate = .pred) %>%
    mutate(model = "Linear Regression", data = "Training"),
  augment(knn_fit, new_data = abalone_train) %>%
    abalone_metrics(truth = age, estimate = .pred) %>%
    mutate(model = "KNN (k = 7)", data = "Training")
) %>%
  select(model, data, .metric, .estimate) %>%
  pivot_wider(names_from = .metric, values_from = .estimate) %>%
  arrange(data, model)
```

```
## # A tibble: 4 x 5
##   model             data       rsq  rmse   mae
```

```
##   <chr>             <chr>    <dbl> <dbl> <dbl>
## 1 KNN (k = 7)       Testing  0.483  2.21  1.62
## 2 Linear Regression Testing  0.585  1.95  1.44
## 3 KNN (k = 7)       Training 0.811  1.46  1.02
## 4 Linear Regression Training 0.552  2.18  1.57
```

On testing data the Linear model outperformed, having a higher R^2 and lower RMSE and MAE. I would assume this is due to KNN relying on being able to see the answers to the training set, not allowing it to generalize on new data. KNN in testing falls behind, where the relationship found in the linear model during training holds up well to new data. I was initially surprised that KNN outperformed in training but Linear outperformed on testing, since I expected the better training model to also do better on testing. Looking at the numbers, though, this is indicative of overfitting. When KNN predicts the training set, each point counts itself as one of its 7 neighbors, so training R^2 is inflated and falls to 0.483 on new data. The less flexible linear model scores about the same on both, 0.552 training vs. 0.585 testing, so training performance alone doesn't show which model will generalize better.